

Reviewer Pack — Patience-Sort LIS of XOR-Lifted Row Permutations Bounds CC^D

Entry dd4c952aa3ea · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-18 09:23:58 UTC

Patience-Sort LIS of XOR-Lifted Row Permutations Bounds CC^D

Entry ID: dd4c952aa3ea **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-18 09:23:58 UTC

1. Conjecture

Field A (mathematical branch): Robinson-Schensted-Knuth / Vershik-Kerov Plancherel limit-shape combinatorics — the longest-increasing-subsequence functional $\lambda_1(w)$ on integer words, computed in $O(|w| \log |w|)$ by patience sorting (Knuth 1970; Logan-Shepp 1977; Vershik-Kerov 1977; Baik-Deift-Johansson 1999 Tracy-Widom fluctuations; Romik 2015). An arXiv search for ‘longest increasing subsequence’ AND (‘lifting theorem’ OR ‘communication complexity’ OR ‘XOR-function’) returns 0 direct hits and <5 adjacent papers (Plancherel-measure work in random-matrix theory, never on lifted communication matrices). The LIS functional uses only comparison/sort (a non-ring partial-order operation), so it does NOT extend to a polynomial extension of the Boolean ring and is safe from Aaronson-Wigderson algebrization. Distinct from blacklisted Tamari/Palio rank (a tree-order statistic), Gromov 4-point δ (a metric invariant), and Burau trace (a representation-theoretic invariant). **Field B** (complexity object): Deterministic two-party communication complexity of XOR-gadget-lifted Boolean functions $M_{\{f \circ \text{XOR}_k\}}(a,b) = f(a \oplus b)$, in the Raz-McKenzie / Göös-Pitassi-Watson / Hatami-Hosseini-Lovett lifting framework where CC^D of XOR-lifts is the canonical Log-Rank Conjecture testbed and the prototypical query-to-communication target.

Statement:

For Boolean $f: \{0,1\}^k \rightarrow \{0,1\}$ with on-set $S(f) = f^{-1}(1)$, $|S(f)| \geq 2$, define for each $a \in \{0,1\}^k$ the row-permutation word $\tau_a = (b \oplus a : b \in S(f), b \text{ listed in lex order})$, and set $\mu(f) = \max_{a \in \{0,1\}^k} \text{LIS}(\tau_a)$, where LIS is the strictly-increasing-subsequence length via patience sorting. Conjecture: for every such f , $\mu(f) \leq \text{rank}_{\{F_2\}}(M_{\{f \circ \text{XOR}_k\}}) + 1$, and consequently $\lceil \log_2 \mu(f) \rceil \leq CC^D(M_{\{f \circ \text{XOR}_k\}}) + 1$ via the canonical algebraic lower bound. A single Boolean f with $\mu(f) \geq \text{rank}_{\{F_2\}}(M_{\{f \circ \text{XOR}_k\}}) + 2$ refutes the conjecture.

Rationale (proposer’s reasoning):

XOR-lifting embeds the additive $(F_2^k, \oplus)$ structure of f into a $2^{k \times 2}k$ communication matrix whose rows are shifts of the indicator of $S(f)$. Vershik-Kerov asserts that random permutations of length n have $\text{LIS} \sim 2\sqrt{n}$; structured shifts (affine $S(f)$, low-rank M) align entries into long monotone runs (large LIS), while ‘rigid’ Boolean functions (e.g., inner product, parity-twist) yield close-to-random permutations with bounded LIS. Patience sorting probes precisely the increasing-monotone substructure that F_2 -rank misses linearly, so a non-trivial μ -vs-rank inequality would expose new XOR-lifting structure — and any

counterexample is a witness to RSK NOT respecting the XOR-orbit structure, falsifying the conjecture in one shot.

Taxonomy category: LIFTING (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): b99755060f581512

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across 30 seeds $\times$ (50 random + 4 structured) Boolean functions for each $k \in \{3, 4, 5\}$ (~ 4860 instances total), compute $\rho(f) = \mu(f) / (\text{rank}_{\{F_2\}}(M_{\{f \circ \text{XOR}_k\}}) + 1)$. SUPPORTED iff every instance satisfies $\rho(f) \leq 1$ (max $\rho \leq 1.0$ exactly); a single instance with $\rho(f) > 1$ FAILS.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.92	SAFE	SAFE
ALGEBRIZATION	SAFE	0.86	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - longest increasing subsequence AND (communication complexity OR log-rank conjecture) - patience sorting AND XOR lifting AND Boolean function-Plancherel measure AND communication complexity AND XOR-function rank

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import itertools

def xor_lift(f, a):
    return tuple((b ^ a) for b in f)

def gaussian_elimination(M):
    n = len(M)
    for i in range(n):
        # Find pivot
        max_row = i
        for j in range(i+1, n):
```

```

        if abs(M[j][i]) > abs(M[max_row][i]):
            max_row = j
        M[i], M[max_row] = M[max_row], M[i]

    # Eliminate below
    for j in range(i+1, n):
        factor = M[j][i] / M[i][i]
        for k in range(n):
            M[j][k] -= factor * M[i][k]

rank = 0
for row in M:
    if any(row):
        rank += 1
return rank

def patience_sort(perm):
    piles = []
    for x in perm:
        inserted = False
        for pile in piles:
            if not pile or pile[-1] < x:
                pile.append(x)
                inserted = True
                break
        if not inserted:
            piles.append([x])
    return len(piles)

def run_trial(seed: int) -> dict:
    random.seed(seed)

    k_values = [3, 4, 5]
    instances_tested = 0
    violations = 0
    max_rho = 0.0

    for n in k_values:
        M_all = []
        S_all = []

        # Generate random Boolean functions
        for _ in range(50):
            f = tuple(random.randint(0, 1) for _ in range(2**n))
            S = [i for i, bit in enumerate(f) if bit == 1]
            if len(S) < 2**(n-1) - 2 or len(S) > 2**(n-1) + 2:
                continue
            M_all.append(M)
            S_all.append(S)
            instances_tested += 1

        # Generate structured stress cases
        for _ in range(4):
            f = tuple(0 if i % 2 == 0 else 1 for i in range(2**n))
            M_all.append(M)
            S_all.append([i for i, bit in enumerate(f) if bit == 1])

```

```

        instances_tested += 1

    for M, S in zip(M_all, S_all):
        for a in itertools.product([0, 1], repeat=n):
            tau_a = tuple((b ^ a) for b in f)
            lis_length = patience_sort(tau_a)
            rho = lis_length / (gaussian_elimination(M) + 1)
            if rho > max_rho:
                max_rho = rho
            if rho > 1.0:
                violations += 1

    conjecture_holds = max_rho <= 1.0
    counterexample = "" if max_rho <= 1.0 else f"rho={max_rho:.4f}"

    return {
        "metric_name": "rho",
        "metric_value": max_rho,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2**i + 3 for i in range(5, 8)]

    results = []
    for seed in seeds:
        trial = run_trial(seed)
        print(f"TRIAL: {trial}")
        results.append(trial)

    mean_rho = sum(r["metric_value"] for r in results) / len(results)
    max_rho = max(r["metric_value"] for r in results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_rho:.4f} std=0.0000 support_fraction=1.0000")
    elif max_rho > 1.0:
        first_failing_seed = next(seed for seed, r in enumerate(results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='rho={max_rho:.4f}' first_failing_seed={first_failing_seed}")
    else:
        print(f"RESULT: INCONCLUSIVE reason=not_enough_evidence")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a979fd53.py", line 111, in <module>
    trial = run_trial(seed)

```

```

^^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a979fd53.py", line 73, in run_trial
    M_all.append(M)
    ^

```

UnboundLocalError: cannot access local variable 'M' where it is not associated with a value

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with an UnboundLocalError before producing any data, so no instances were evaluated and the pre-registered support condition cannot be assessed. | next: Fix the UnboundLocalError around line 73 (ensure M is always assigned before append, likely by initializing M or handling the branch where it isn't computed) and rerun the full 30 seeds $\times$ $k \in \{3,4,5\}$ sweep.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	283544
2	preregistration	claude_max	opus	0	0	5751
3	novelty	claude_max	opus	0	0	4666
4	novelty	claude_max	opus	0	0	8035
5	test_gen	mistral	codestral-latest	0	0	322378
6	test_gen	mistral	codestral-latest	0	0	319891
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	269700
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	289325
9	judge	claude_max	opus	0	0	4243

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 1507531 ms total latency. Provider mix: {'claude_max': 5, 'mistral': 2, 'ollama_remote': 2}

(full prompt+response transcripts available in `research/audit/dd4c952aa3ea.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/dd4c952aa3ea.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/dd4c952aa3ea.tar.gz (if generated)